2026-04-26-jackett-autoconfig-clean-rebuild

Jackett Auto-Config + Clean-Slate Rebuild — Implementation Plan

Revision: 1 **Last modified:** 2026-06-06T00:00:00Z

> **Status (reconciled 2026-04-27):** All 84 checkboxes marked `[x]`. The auto-config module shipped via commits `45591c2` (Layer 1), `9e70cc1` (Layer 2), `5f12be0` (Layers 3-7), `186d26a` (catalog/template fix), `fc1f009` (parity audit + docs). The successor plan (`2026-04-27-jackett-management-ui-and-system-db.md`) supersedes the Python `/api/v1/jackett/autoconfig/last` endpoint with the Go `boba-jackett:7189` service (autoconfig runs history at `/api/v1/jackett/autoconfig/runs`); the corresponding Python file `download-proxy/src/api/jackett.py` was removed in commit `<see Task 38 of successor plan>`. The functional intent of every step in this plan has shipped.

> **For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (`- [ ]`) syntax for tracking.

Goal: Implement Jackett indexer auto-configuration that discovers `<NAME>_USERNAME/_PASSWORD/_COOKIES` env triples and idempotently configures matching indexers in Jackett at proxy startup; verify end-to-end via clean-slate container rebuild and the seven CONSTITUTION-mandated test layers.

Architecture: New async module `merge_service/jackett_autoconfig.py` exports a single `autoconfigure_jackett()` coroutine. Called from FastAPI `lifespan()` after Jackett's healthcheck passes. Reads env, fuzzy-matches tracker names

against Jackett's catalog, idempotently posts indexer config. Read-only result endpoint `GET /api/v1/jackett/autoconfig/last` exposes the redacted last-run summary. Failure never blocks boot.

Tech Stack: Python 3.12, FastAPI, aiohttp, Pydantic, `Levenshtein`, `tenacity`, `pytest`, `pytest-benchmark`, `schemathesis`, `bandit`. All existing project deps — no new runtime additions.

Spec: `docs/superpowers/specs/2026-04-26-jackett-autoconfig-clean-rebuild-design.md` (commit `3e76f77`).

Minor deviation from spec: endpoint path `/merge/jackett/autoconfig/last` → `/api/v1/jackett/autoconfig/last` to match existing router conventions in `api/__init__.py`.

Conventions Used Throughout

- All `pytest` invocations are wrapped: `nice -n 19 ionice -c 3 -p 1 python3 -m pytest <args> --import-mode=importlib`.
- All `pytest` runs use `-x --maxfail=1 -p no:randomly` for deterministic per-task fail-fast (still uses `pytest-randomly` for ordering within a single run).
- Per-layer push: after **every test in a layer** is green, run `git push origin main && git push github main && git push upstream main` (all three URLs are identical; we push all three so refs match locally).
- Hard stops: layer fail after one fix attempt → halt; push reject → halt; healthcheck unhealthy 3 min → halt + dump logs; bench regression > 2× baseline → halt.
- Container runtime auto-detect — use `podman` (preferred per project) but `docker` works as a fallback; commands below show `podman`.

Phase 0: Pre-Flight & Tear-Down

Task 0.1: Verify clean working tree and required env

Files: none (read-only check)

- [ ] **Step 1: Confirm clean tree**

```
cd /run/media/milosvasic/DATA4TB/Projects/Boba
git status --porcelain
```

Expected: empty output (no uncommitted changes). If non-empty, halt and ask the operator.

- [ ] **Step 2: Confirm `.env` has the four credential triples**

```
grep -E "^(RUTRACKER|KINOZAL|NNMCLUB|IPTORRENTS)_(USERNAME|
        PASSWORD|COOKIES)=" .env | wc -l
```

Expected: `>= 4` (each tracker contributes at least one line). If `0`, halt — auto-config has nothing to discover.

- [ ] **Step 3: Confirm Jackett API key placeholder is wired**

```
grep -E "^JACKETT_API_KEY=" .env || echo "missing"
```

If output is `missing`, append a placeholder:
`echo 'JACKETT_API_KEY=' >> .env`. (`start.sh` populates it from Jackett at boot; an empty value is fine pre-boot.)

- [ ] **Step 4: Record current commit SHA for the parity audit anchor**

```
git rev-parse HEAD > /tmp/jackett-autoconfig-baseline-sha.txt
cat /tmp/jackett-autoconfig-baseline-sha.txt
```

This SHA is referenced in `PARITY_GAPS.md` later.

Task 0.2: Tear down stack + wipe Jackett state

Files: none (operates on running containers + `./config/jackett`)

- [ ] **Step 1: Stop both profiles defensively**

```
podman compose --profile go down --remove-orphans 2>/dev/null ||
        true
podman compose down --remove-orphans
```

Expected: containers `qbittorrent`, `jackett`, `qbittorrent-proxy` (and Go variant if it was running) stopped and removed. Trailing `||` `true` on the Go line because the Go profile may not be running.

- [ ] **Step 2: Remove project-built images**

```
podman image rm -f \
  $(podman images --filter reference='*qbittorrent-proxy*' --
        filter reference='*qbitorrent-proxy-go*' -q) \
```

```
  2>/dev/null || true
podman image prune -f
```

- [ ] **Step 3: Wipe Jackett state only**

```
rm -rf ./config/jackett
```

Expected: `./config/jackett` no longer exists. **Do not** touch `./config/qBittorrent`, `./tmp`, or `/mnt/DATA`.

- [ ] **Step 4: Verify wipe boundaries**

```
test ! -d ./config/jackett && echo "jackett wiped: OK"
test -d ./config/qBittorrent && echo "qbittorrent preserved: OK"
test -d ./tmp && echo "tmp preserved: OK"
test -d /mnt/DATA && echo "data preserved: OK"
```

Expected: all four lines print `OK`. If any fails, halt.

Phase 1: Implement Auto-Config Module (Layer 1: Unit Tests)

We build the module bottom-up: data model → env scanner → fuzzy matcher → idempotency check → orchestrator → wiring → endpoint. Each task is TDD: failing test → minimal code → passing test → local commit. Push happens once after all unit tests pass.

Task 1.1: AutoconfigResult Pydantic model + redaction guarantee

Files: - Create: `download-proxy/src/merge_service/jackett_autoconfig.py` - Create: `tests/unit/merge_service/test_jackett_autoconfig.py`

- [ ] **Step 1: Write failing test for `AutoconfigResult` shape and redaction**

```
# tests/unit/merge_service/test_jackett_autoconfig.py
import os
import sys

# Match the existing path-injection convention used across tests/
        unit/.
sys.path.insert(
    0,
```

```
    os.path.join(os.path.dirname(__file__), "..", "..", "..",
        "download-proxy", "src"),
)

import json
from datetime import datetime, timezone

import pytest


def test_autoconfig_result_serializes_with_no_credentials():
    from merge_service.jackett_autoconfig import AutoconfigResult

    r = AutoconfigResult(
        ran_at=datetime(2026, 4, 26, 14, 23, 11,
        tzinfo=timezone.utc),
        discovered_credentials=["RUTRACKER", "KINOZAL"],
        matched_indexers={"RUTRACKER": "rutracker", "KINOZAL":
        "kinozalbiz"},
        configured_now=["rutracker"],
        already_present=["kinozalbiz"],
        skipped_no_match=[],
        skipped_ambiguous=[],
        errors=[],
    )

    body = r.model_dump_json()
    parsed = json.loads(body)

    # Required fields.
    assert parsed["discovered"] == ["RUTRACKER", "KINOZAL"]
    assert parsed["configured_now"] == ["rutracker"]
    assert parsed["already_present"] == ["kinozalbiz"]
    assert parsed["ran_at"].startswith("2026-04-26T14:23:11")

    # No credential value, anywhere.
    assert "password" not in body.lower()
    assert "username" not in body.lower()
    assert "cookie" not in body.lower()


def test_autoconfig_result_repr_excludes_credentials():
    from merge_service.jackett_autoconfig import AutoconfigResult

    r = AutoconfigResult(
        ran_at=datetime.now(timezone.utc),
        discovered_credentials=["RUTRACKER"],
        matched_indexers={"RUTRACKER": "rutracker"},
        configured_now=["rutracker"],
        already_present=[],
        skipped_no_match=[],
        skipped_ambiguous=[],
```

```
        errors=[],
    )
    txt = repr(r)
    assert "password" not in txt.lower()
    assert "secret" not in txt.lower()
```

▣

Step 2: Run the test — confirm it fails because module doesn't exist

```
nice -n 19 ionice -c 3 -p 1 python3 -m pytest tests/unit/
        merge_service/test_jackett_autoconfig.py -v --import-
        mode=importlib
```

Expected: `ModuleNotFoundError: No module named 'merge_service.jackett_autoconfig'` (FAIL).

▣

Step 3: Create the module with the data model

```
# download-proxy/src/merge_service/jackett_autoconfig.py
"""Jackett auto-configuration at proxy startup.

Discovers <NAME>_USERNAME/_PASSWORD/_COOKIES env triples and
idempotently configures matching indexers in Jackett. Failure
never raises out of autoconfigure_jackett(); all errors are
captured in AutoconfigResult.errors.
"""

from __future__ import annotations

from datetime import datetime
from typing import Any

from pydantic import BaseModel, Field


class AmbiguousMatch(BaseModel):
    env_name: str
    candidates: list[str]


class AutoconfigResult(BaseModel):
    ran_at: datetime
    discovered_credentials: list[str] =
        Field(default_factory=list, alias="discovered")
    matched_indexers: dict[str, str] = Field(default_factory=dict)
    configured_now: list[str] = Field(default_factory=list)
    already_present: list[str] = Field(default_factory=list)
    skipped_no_match: list[str] = Field(default_factory=list)
    skipped_ambiguous: list[AmbiguousMatch] =
        Field(default_factory=list)
    errors: list[str] = Field(default_factory=list)
```

```
    model_config = {"populate_by_name": True}

    def __repr__(self) -> str:
        return (
            f"AutoconfigResult(ran_at={self.ran_at.isoformat()}, "
            f"discovered={len(self.discovered_credentials)}, "
            f"configured_now={len(self.configured_now)}, "
            f"errors={len(self.errors)})"
        )
```

☐ **Step 4: Run the test — expect PASS**

```
nice -n 19 ionice -c 3 -p 1 python3 -m pytest tests/unit/
        merge_service/test_jackett_autoconfig.py -v --import-
        mode=importlib
```

Expected: 2 passed.

☐ **Step 5: Commit (local)**

```
git add download-proxy/src/merge_service/jackett_autoconfig.py
        tests/unit/merge_service/test_jackett_autoconfig.py
git commit -m "feat(jackett_autoconfig): AutoconfigResult model
        with redacted serialization"
```

Task 1.2: Env scanner with denylist

Files: - Modify: `download-proxy/src/merge_service/jackett_autoconfig.py` - Modify: `tests/unit/merge_service/test_jackett_autoconfig.py`

☐ **Step 1: Append failing tests for the env scanner**

```
# Append to tests/unit/merge_service/test_jackett_autoconfig.py
def test_env_scan_finds_username_password_pair():
    from merge_service.jackett_autoconfig import
        _scan_env_credentials

    env = {
        "RUTRACKER_USERNAME": "u",
        "RUTRACKER_PASSWORD": "p",
        "PATH": "/usr/bin",
    }
    bundles = _scan_env_credentials(env, exclude=set())
    assert "RUTRACKER" in bundles
    assert bundles["RUTRACKER"]["username"] == "u"
    assert bundles["RUTRACKER"]["password"] == "p"
```

```
def test_env_scan_finds_cookies_alone():
    from merge_service.jackett_autoconfig import
        _scan_env_credentials

    env = {"NNMCLUB_COOKIES": "abc=def; xyz=uvw"}
    bundles = _scan_env_credentials(env, exclude=set())
    assert "NNMCLUB" in bundles
    assert bundles["NNMCLUB"]["cookies"] == "abc=def; xyz=uvw"


def test_env_scan_skips_incomplete_bundle_silently():
    from merge_service.jackett_autoconfig import
        _scan_env_credentials

    env = {"RUTRACKER_USERNAME": "u"}  # no PASSWORD, no COOKIES
    bundles = _scan_env_credentials(env, exclude=set())
    assert bundles == {}


def test_env_scan_respects_denylist():
    from merge_service.jackett_autoconfig import
        _scan_env_credentials

    env = {
        "QBITTORRENT_USERNAME": "admin",
        "QBITTORRENT_PASSWORD": "admin",
        "RUTRACKER_USERNAME": "u",
        "RUTRACKER_PASSWORD": "p",
    }
    bundles = _scan_env_credentials(env, exclude={"QBITTORRENT"})
    assert "QBITTORRENT" not in bundles
    assert "RUTRACKER" in bundles


def test_env_scan_ignores_lowercase_and_irrelevant_suffixes():
    from merge_service.jackett_autoconfig import
        _scan_env_credentials

    env = {
        "rutracker_username": "u",
        # lowercase — not a credential per regex
        "RUTRACKER_USER": "u",  # _USER (not _USERNAME) — not a
        match
        "DEBUG_PASSWORD": "x",  # only password, no username —
        incomplete
    }
    bundles = _scan_env_credentials(env, exclude=set())
    assert bundles == {}
```

Step 2: Run new tests — confirm FAIL with `_scan_env_credentials` undefined

```
nice -n 19 ionice -c 3 -p 1 python3 -m pytest tests/unit/
        merge_service/test_jackett_autoconfig.py -v --import-
        mode=importlib
```

Expected: `AttributeError` / `ImportError` for `_scan_env_credentials`.

▣

Step 3: Add the env scanner to the module

Insert near the top of `download-proxy/src/merge_service/jackett_autoconfig.py`, after the imports:

```
import re
from collections.abc import Mapping

# Captures: <NAME>_USERNAME, <NAME>_PASSWORD, <NAME>_COOKIES
_CRED_RE = re.compile(r"^([A-Z][A-Z0-9_]+?)_(USERNAME|PASSWORD|
        COOKIES)$")

DEFAULT_EXCLUDE = frozenset(
    {"QBITTORRENT", "JACKETT", "WEBUI", "PROXY", "MERGE",
        "BRIDGE"}
)


def _scan_env_credentials(
    env: Mapping[str, str],
    exclude: frozenset[str] | set[str],
) -> dict[str, dict[str, str]]:
    """Scan env for tracker credential triples.

    Returns a dict keyed by tracker name, where each value
        contains
    the keys {"username", "password"} or {"cookies"}. Bundles that
    are not "complete" (no usable credentials) are dropped
        silently.
    """
    grouped: dict[str, dict[str, str]] = {}
    for key, value in env.items():
        m = _CRED_RE.match(key)
        if not m:
            continue
        name, kind = m.group(1), m.group(2)
        if name in exclude:
            continue
        bucket = grouped.setdefault(name, {})
        bucket[kind.lower()] = value

    complete: dict[str, dict[str, str]] = {}
    for name, fields in grouped.items():
```

```
        has_userpass = "username" in fields and "password" in
        fields
        has_cookies = "cookies" in fields
        if has_userpass or has_cookies:
            complete[name] = fields
    return complete
```

☐ **Step 4: Run all tests in the file — expect 7 passed**

```
nice -n 19 ionice -c 3 -p 1 python3 -m pytest tests/unit/
        merge_service/test_jackett_autoconfig.py -v --import-
        mode=importlib
```

Expected: 7 passed.

☐ **Step 5: Commit (local)**

```
git add download-proxy/src/merge_service/jackett_autoconfig.py
        tests/unit/merge_service/test_jackett_autoconfig.py
git commit -m "feat(jackett_autoconfig): env scanner with
        denylist"
```

Task 1.3: Fuzzy matcher with override precedence

Files: - Modify: `download-proxy/src/merge_service/jackett_autoconfig.py` - Modify: `tests/unit/merge_service/test_jackett_autoconfig.py`

☐ **Step 1: Append failing tests for fuzzy matching**

```
# Append to tests/unit/merge_service/test_jackett_autoconfig.py
def test_fuzzy_match_exact_name_returns_indexer():
    from merge_service.jackett_autoconfig import _match_indexers

    catalog = [{"id": "rutracker", "name": "RuTracker"}, {"id":
        "kinozalbiz", "name": "KinoZal"}]
    bundles = {"RUTRACKER": {"username": "u", "password": "p"}}
    matched, ambiguous, unmatched = _match_indexers(bundles,
        catalog, override={})
    assert matched == {"RUTRACKER": "rutracker"}
    assert ambiguous == []
    assert unmatched == []


def test_fuzzy_match_below_threshold_goes_to_unmatched():
    from merge_service.jackett_autoconfig import _match_indexers

    catalog = [{"id": "demonoid", "name": "Demonoid"}]
    bundles = {"RUTRACKER": {"username": "u", "password": "p"}}
```

```
    matched, ambiguous, unmatched = _match_indexers(bundles,
        catalog, override={})
    assert matched == {}
    assert unmatched == ["RUTRACKER"]


def test_fuzzy_match_ambiguous_records_candidates():
    from merge_service.jackett_autoconfig import _match_indexers

    # Two equally-close-to-"NNMCLUB" indexer names → ambiguous
    catalog = [
        {"id": "nnmclub", "name": "NNMClub"},
        {"id": "nnmcluborg", "name": "NNMClubOrg"},
    ]
    bundles = {"NNMCLUB": {"cookies": "x"}}
    matched, ambiguous, unmatched = _match_indexers(bundles,
        catalog, override={})
    # Either the first wins outright, or both get flagged
        ambiguous;
    # algorithm picks deterministically — test the contract: at
        most
    # one wins, and remainder go to ambiguous.
    if matched:
        assert "NNMCLUB" in matched
    else:
        assert any(a.env_name == "NNMCLUB" for a in ambiguous)


def test_override_takes_precedence_over_fuzzy_match():
    from merge_service.jackett_autoconfig import _match_indexers

    catalog = [
        {"id": "rutracker", "name": "RuTracker"},
        {"id": "rutrackerme", "name": "RutrackerMe"},
    ]
    bundles = {"RUTRACKER": {"username": "u", "password": "p"}}
    matched, _, _ = _match_indexers(bundles, catalog,
        override={"RUTRACKER": "rutrackerme"})
    assert matched == {"RUTRACKER": "rutrackerme"}


def test_override_to_unknown_id_records_error():
    from merge_service.jackett_autoconfig import _match_indexers

    catalog = [{"id": "rutracker", "name": "RuTracker"}]
    bundles = {"RUTRACKER": {"username": "u", "password": "p"}}
    matched, _, _ = _match_indexers(
        bundles, catalog, override={"RUTRACKER": "does-not-exist"}
    )
    # Override pointing to unknown id should fall back to fuzzy
        match.
    assert matched == {"RUTRACKER": "rutracker"}
```

▣ Step 2: Run — expect FAIL

```
nice -n 19 ionice -c 3 -p 1 python3 -m pytest tests/unit/
        merge_service/test_jackett_autoconfig.py -v --import-
        mode=importlib
```

Expected: `_match_indexers` undefined.

▣ Step 3: Add the matcher

Append to `download-proxy/src/merge_service/jackett_autoconfig.py`:

```
import Levenshtein

FUZZY_THRESHOLD = 0.85


def _match_indexers(
    bundles: dict[str, dict[str, str]],
    catalog: list[dict[str, Any]],
    override: dict[str, str],
) -> tuple[dict[str, str], list[AmbiguousMatch], list[str]]:
    """Map env tracker names to Jackett indexer ids.

    Returns (matched, ambiguous, unmatched).
    Override mappings take precedence; fall back to fuzzy match
    on indexer id and indexer display name (case-insensitive).
    """
    catalog_ids = {entry["id"] for entry in catalog}
    matched: dict[str, str] = {}
    ambiguous: list[AmbiguousMatch] = []
    unmatched: list[str] = []

    for env_name in bundles:
        # Override path — only honored if the target id actually
        exists.
        target = override.get(env_name)
        if target and target in catalog_ids:
            matched[env_name] = target
            continue

        # Fuzzy path — score against id AND name; take the best
        per indexer.
        scored: list[tuple[str, float]] = []
        needle = env_name.lower()
        for entry in catalog:
            id_score = Levenshtein.ratio(needle,
        entry["id"].lower())
            name_score = Levenshtein.ratio(needle,
        entry["name"].lower())
            best = max(id_score, name_score)
```

```
            if best >= FUZZY_THRESHOLD:
                scored.append((entry["id"], best))

        if not scored:
            unmatched.append(env_name)
            continue

        scored.sort(key=lambda t: (-t[1], t[0]))  # high score,
        then id alpha
        top_score = scored[0][1]
        # Only the strictly-top scorer wins; ties → ambiguous.
        ties = [sid for sid, sc in scored if sc == top_score]
        if len(ties) == 1:
            matched[env_name] = ties[0]
        else:
            ambiguous.append(AmbiguousMatch(env_name=env_name,
        candidates=ties))

    return matched, ambiguous, unmatched
```

☐ **Step 4: Run — expect 12 passed**

```
nice -n 19 ionice -c 3 -p 1 python3 -m pytest tests/unit/
        merge_service/test_jackett_autoconfig.py -v --import-
        mode=importlib
```

Expected: 12 passed.

☐ **Step 5: Commit (local)**

```
git add download-proxy/src/merge_service/jackett_autoconfig.py
        tests/unit/merge_service/test_jackett_autoconfig.py
git commit -m "feat(jackett_autoconfig): fuzzy matcher with
        override precedence"
```

Task 1.4: Indexer config POST + idempotency check

Files: - Modify: `download-proxy/src/merge_service/jackett_autoconfig.py` - Modify: `tests/unit/merge_service/test_jackett_autoconfig.py`

☐ **Step 1: Append failing tests using `unittest.mock.AsyncMock`**

```
# Append to tests/unit/merge_service/test_jackett_autoconfig.py
from unittest.mock import AsyncMock, MagicMock
```

```
def _mock_aiohttp_response(status: int, json_data: Any = None,
        text: str = ""):
    """Build an aiohttp-style async context-manager response.

    Mirrors the pattern in tests/unit/
        test_private_tracker_search.py.
    """
    resp = MagicMock()
    resp.status = status
    resp.json = AsyncMock(return_value=json_data)
    resp.text = AsyncMock(return_value=text)
    cm = MagicMock()
    cm.__aenter__ = AsyncMock(return_value=resp)
    cm.__aexit__ = AsyncMock(return_value=False)
    return cm


@pytest.mark.asyncio
async def test_idempotency_skips_already_configured():
    from merge_service.jackett_autoconfig import _configure_one

    session = MagicMock()
    # /indexers (already-configured) returns rutracker — should
        skip.
    session.get = MagicMock(
        return_value=_mock_aiohttp_response(
            200,
            json_data=[{"id": "rutracker"}],
        )
    )
    session.post = MagicMock()

    result = await _configure_one(
        session,
        jackett_url="http://jackett:9117",
        api_key="fake",
        indexer_id="rutracker",
        creds={"username": "u", "password": "p"},
        already_configured={"rutracker"},
    )
    assert result == ("already_present", None)
    session.post.assert_not_called()


@pytest.mark.asyncio
async def test_configure_posts_template_with_filled_credentials():
    from merge_service.jackett_autoconfig import _configure_one

    template = {
        "config": [
            {"id": "username", "value": ""},
            {"id": "password", "value": ""},
```

```
        ]
    }
    session = MagicMock()

    config_get_cm = _mock_aiohttp_response(200,
        json_data=template)
    config_post_cm = _mock_aiohttp_response(200, json_data={"ok":
        True})

    session.get = MagicMock(return_value=config_get_cm)
    session.post = MagicMock(return_value=config_post_cm)

    result = await _configure_one(
        session,
        jackett_url="http://jackett:9117",
        api_key="fake",
        indexer_id="rutracker",
        creds={"username": "u", "password": "p"},
        already_configured=set(),
    )
    assert result == ("configured", None)
    session.post.assert_called_once()
    # Verify credentials made it into the POST body.
    posted_kwargs = session.post.call_args.kwargs
    payload = posted_kwargs.get("json") or {}
    config = payload.get("config", [])
    fields = {f["id"]: f["value"] for f in config}
    assert fields["username"] == "u"
    assert fields["password"] == "p"


@pytest.mark.asyncio
async def test_configure_records_4xx_as_error_no_retry():
    from merge_service.jackett_autoconfig import _configure_one

    template = {"config": [{"id": "username", "value": ""}]}
    session = MagicMock()
    session.get =
        MagicMock(return_value=_mock_aiohttp_response(200,
        json_data=template))
    session.post =
        MagicMock(return_value=_mock_aiohttp_response(401,
        text="bad creds"))

    result = await _configure_one(
        session,
        jackett_url="http://jackett:9117",
        api_key="fake",
        indexer_id="rutracker",
        creds={"username": "u"},
        already_configured=set(),
    )
    status, err = result
```

```
    assert status == "error"
    assert err and "401" in err
```

☐ **Step 2: Run — expect FAIL** (`_configure_one` undefined)

```
nice -n 19 ionice -c 3 -p 1 python3 -m pytest tests/unit/
        merge_service/test_jackett_autoconfig.py -v --import-
        mode=importlib
```

☐ **Step 3: Add `_configure_one` to the module**

Append to `download-proxy/src/merge_service/jackett_autoconfig.py`:

```
import asyncio
from typing import Literal

import aiohttp


async def _configure_one(
    session: aiohttp.ClientSession,
    *,
    jackett_url: str,
    api_key: str,
    indexer_id: str,
    creds: dict[str, str],
    already_configured: set[str],
) -> tuple[Literal["configured", "already_present", "error"], str
        | None]:
    """Configure a single Jackett indexer.

    Returns a status tuple. Never raises out of its boundary.
    """
    if indexer_id in already_configured:
        return ("already_present", None)

    headers = {"Accept": "application/json"}
    params = {"apikey": api_key}

    # 1. Fetch config template for the indexer.
    cfg_url = f"{jackett_url}/api/v2.0/indexers/{indexer_id}/
        config"
    try:
        async with session.get(cfg_url, params=params,
        headers=headers) as resp:
            if resp.status >= 400:
                return ("error", f"template fetch HTTP
        {resp.status}")
            template = await resp.json()
    except (aiohttp.ClientError, asyncio.TimeoutError) as e:
```

```
        return ("error", f"template fetch network error:
        {type(e).__name__}")

    # 2. Map credential fields onto template.
    config_fields = template.get("config", [])
    field_map = {
        "username": creds.get("username"),
        "password": creds.get("password"),
        "cookieheader": creds.get("cookies"),
        "cookies": creds.get("cookies"),
    }
    populated: list[dict[str, Any]] = []
    for field in config_fields:
        fid = field.get("id")
        if fid in field_map and field_map[fid] is not None:
            populated.append({"id": fid, "value": field_map[fid]})
        else:
            populated.append(field)

    # 3. POST the populated config (one retry on 5xx).
    body = {"config": populated}
    for attempt in (1, 2):
        try:
            async with session.post(
                cfg_url, params=params, headers=headers, json=body
            ) as resp:
                if resp.status >= 500 and attempt == 1:
                    await asyncio.sleep(2.0)
                    continue
                if resp.status >= 400:
                    return ("error", f"config POST HTTP
        {resp.status}")
                return ("configured", None)
        except (aiohttp.ClientError, asyncio.TimeoutError) as e:
            if attempt == 1:
                await asyncio.sleep(2.0)
                continue
            return ("error", f"config POST network error:
        {type(e).__name__}")
    return ("error", "config POST exhausted retries")
```

Keep per-function `@pytest.mark.asyncio` decorators on each async test. Do **not** set a module-level `pytestmark = pytest.mark.asyncio` — Tasks 1.1-1.3 wrote sync tests in this same file, and module-level marking would break them.

Step 4: Run — expect 15 passed

```
nice -n 19 ionice -c 3 -p 1 python3 -m pytest tests/unit/
        merge_service/test_jackett_autoconfig.py -v --import-
        mode=importlib
```

Expected: 15 passed.

▣

Step 5: Commit (local)

```
git add download-proxy/src/merge_service/jackett_autoconfig.py
        tests/unit/merge_service/test_jackett_autoconfig.py
git commit -m "feat(jackett_autoconfig): per-indexer configure
        with idempotency + retry"
```

Task 1.5: Top-level orchestrator + error capture + 60s ceiling

Files: - Modify: `download-proxy/src/merge_service/jackett_autoconfig.py` - Modify: `tests/unit/merge_service/test_jackett_autoconfig.py`

▣

Step 1: Append failing tests

```
# Append to tests/unit/merge_service/test_jackett_autoconfig.py
@pytest.mark.asyncio
async def
        test_autoconfigure_jackett_unreachable_returns_error_no_raise():
    from merge_service.jackett_autoconfig import
        autoconfigure_jackett

    # Point at a port nothing is listening on.
    result = await autoconfigure_jackett(
        jackett_url="http://127.0.0.1:1",
        api_key="fake",
        env={"RUTRACKER_USERNAME": "u", "RUTRACKER_PASSWORD":
        "p"},
        timeout=1.0,
    )
    assert any("jackett_unreachable" in e or "network" in
        e.lower() for e in result.errors)
    assert result.configured_now == []


@pytest.mark.asyncio
async def
        test_autoconfigure_jackett_total_timeout_caps_at_60s(monkeypatch):

        """The outer asyncio.wait_for(60s) must not allow the
        function to hang forever."""
    from merge_service.jackett_autoconfig import
        autoconfigure_jackett

    # Force an unreachable URL with very short timeout — function
        must
    # return well under 60s. We assert it returns at all, in <5s.
    import time
```

```
    start = time.monotonic()
    result = await autoconfigure_jackett(
        jackett_url="http://127.0.0.1:1",
        api_key="fake",
        env={},
        timeout=1.0,
    )
    elapsed = time.monotonic() - start
    assert elapsed < 5.0
    assert isinstance(result.ran_at, datetime)
```

☐ **Step 2: Run — expect FAIL** (`autoconfigure_jackett` undefined)

```
nice -n 19 ionice -c 3 -p 1 python3 -m pytest tests/unit/
        merge_service/test_jackett_autoconfig.py -v --import-
        mode=importlib
```

☐ **Step 3: Add the orchestrator**

Append to `download-proxy/src/merge_service/jackett_autoconfig.py`:

```
import logging
import os
from datetime import timezone

logger = logging.getLogger(__name__)

TOTAL_TIMEOUT_SECONDS = 60.0


def _parse_indexer_map(raw: str | None) -> dict[str, str]:
    if not raw:
        return {}
    out: dict[str, str] = {}
    for pair in raw.split(","):
        pair = pair.strip()
        if not pair or ":" not in pair:
            continue
        key, value = pair.split(":", 1)
        key, value = key.strip().upper(), value.strip()
        if key and value:
            out[key] = value
    return out


def _parse_exclude(raw: str | None) -> frozenset[str]:
    if raw is None:
        return DEFAULT_EXCLUDE
```

```
    parts = {p.strip().upper() for p in raw.split(",") if
        p.strip()}
    return frozenset(parts) if parts else DEFAULT_EXCLUDE


async def autoconfigure_jackett(
    jackett_url: str,
    api_key: str,
    env: Mapping[str, str],
    indexer_map_override: dict[str, str] | None = None,
    timeout: float = 10.0,
) -> AutoconfigResult:
    """Discover env credentials, fuzzy-match to Jackett indexers,
        configure them.

    Never raises. All failures land in AutoconfigResult.errors.
    """
    started = datetime.now(timezone.utc)
    # Resolve override: explicit arg wins, else env var.
    override = (
        indexer_map_override
        if indexer_map_override is not None
        else _parse_indexer_map(env.get("JACKETT_INDEXER_MAP"))
    )
    exclude =
        _parse_exclude(env.get("JACKETT_AUTOCONFIG_EXCLUDE"))

    bundles = _scan_env_credentials(env, exclude=exclude)
    discovered = sorted(bundles.keys())

    if not bundles:
        return AutoconfigResult(ran_at=started,
        discovered_credentials=[])

    if not api_key:
        return AutoconfigResult(
            ran_at=started,
            discovered_credentials=discovered,
            errors=["jackett_auth_missing_key"],
        )

    headers = {"Accept": "application/json"}
    params = {"apikey": api_key}
    client_timeout = aiohttp.ClientTimeout(total=timeout)

    try:
        async with asyncio.timeout(TOTAL_TIMEOUT_SECONDS):
            async with
        aiohttp.ClientSession(timeout=client_timeout) as session:
                # Catalog
                catalog: list[dict[str, Any]] = []
                catalog_err: str | None = None
                try:
```

```
            cat_url = f"{jackett_url}/api/v2.0/indexers/
all/results"
            async with session.get(cat_url,
params=params, headers=headers) as r:
                if r.status == 401:
                    catalog_err = "jackett_auth_failed"
                elif r.status >= 400:
                    catalog_err =
f"jackett_catalog_http_{r.status}"
                else:
                    try:
                        data = await r.json()

# Jackett returns either a list or {"Indexers": [...]}
                        if isinstance(data, list):
                            catalog = data
                        elif isinstance(data, dict):
                            catalog =
data.get("Indexers", [])
                        if not isinstance(catalog, list):
                            catalog = []
                            catalog_err =
"catalog_parse_failed"
                    except (aiohttp.ContentTypeError,
ValueError):
                        catalog_err =
"catalog_parse_failed"
        except (aiohttp.ClientError,
asyncio.TimeoutError) as e:
            catalog_err = (
                "jackett_unreachable"
                if isinstance(e,
aiohttp.ClientConnectorError)
                else f"network: {type(e).__name__}"
            )

        if catalog_err:
            return AutoconfigResult(
                ran_at=started,
                discovered_credentials=discovered,
                errors=[catalog_err],
            )

        # Already-configured set
        already: set[str] = set()
        try:
            list_url = f"{jackett_url}/api/v2.0/indexers"
            async with session.get(list_url,
params=params, headers=headers) as r:
                if r.status < 400:
                    data = await r.json()
                    if isinstance(data, list):
                        already = {e.get("id") for e in
data if e.get("id")}
```

```
            except (aiohttp.ClientError,
    asyncio.TimeoutError):
                # Non-fatal: idempotency check failed; we'll
    attempt and
                # tolerate "already-configured" 4xx per
    indexer.
                pass

            matched, ambiguous, unmatched = _match_indexers(
                bundles, catalog, override
            )

            configured_now: list[str] = []
            already_present: list[str] = []
            errors: list[str] = []
            for env_name, indexer_id in matched.items():
                status, err = await _configure_one(
                    session,
                    jackett_url=jackett_url,
                    api_key=api_key,
                    indexer_id=indexer_id,
                    creds=bundles[env_name],
                    already_configured=already,
                )
                if status == "configured":
                    configured_now.append(indexer_id)
                elif status == "already_present":
                    already_present.append(indexer_id)
                else:
                    errors.append(f"indexer_config_failed:
    {indexer_id}:{err}")

            return AutoconfigResult(
                ran_at=started,
                discovered_credentials=discovered,
                matched_indexers=matched,
                configured_now=configured_now,
                already_present=already_present,
                skipped_no_match=unmatched,
                skipped_ambiguous=ambiguous,
                errors=errors,
            )
    except TimeoutError:
        return AutoconfigResult(
            ran_at=started,
            discovered_credentials=discovered,
            errors=["jackett_total_timeout_60s"],
        )
    except Exception as e:  # noqa: BLE001 — explicit boundary
        logger.exception("Unexpected autoconfig failure")
        return AutoconfigResult(
            ran_at=started,
```

```
            discovered_credentials=discovered,
            errors=[f"unexpected: {type(e).__name__}"],
        )
```

☐ **Step 4: Run — expect 17 passed**

```
nice -n 19 ionice -c 3 -p 1 python3 -m pytest tests/unit/
        merge_service/test_jackett_autoconfig.py -v --import-
        mode=importlib
```

Expected: 17 passed.

☐ **Step 5: Lint**

```
ruff check download-proxy/src/merge_service/jackett_autoconfig.py
ruff format --check download-proxy/src/merge_service/
        jackett_autoconfig.py
```

Fix any complaints. Re-run tests.

☐ **Step 6: Commit (local)**

```
git add download-proxy/src/merge_service/jackett_autoconfig.py
        tests/unit/merge_service/test_jackett_autoconfig.py
git commit -m "feat(jackett_autoconfig): top-level orchestrator
        with 60s ceiling"
```

Task 1.6: Wire `autoconfigure_jackett` into FastAPI lifespan

Files: - Modify: `download-proxy/src/api/__init__.py:28-56` (extend `lifespan`)

☐ **Step 1: Edit `lifespan()` to call `autoconfigure_jackett` after services start**

Find the existing `lifespan` block (lines 28-56). After `await app.state.scheduler.start()` and before `logger.info("Merge Service API started")`, insert:

```
    # Jackett auto-configuration — best-effort, never blocks boot.
    from merge_service.jackett_autoconfig import
        autoconfigure_jackett

    jackett_url = os.getenv("JACKETT_URL", "http://
        localhost:9117")
    jackett_key = os.getenv("JACKETT_API_KEY", "")
    if jackett_key and jackett_key != "YOUR_API_KEY_HERE":
        try:
```

```
            app.state.jackett_autoconfig_last = await
        autoconfigure_jackett(
                jackett_url=jackett_url,
                api_key=jackett_key,
                env=os.environ,
            )
            r = app.state.jackett_autoconfig_last
            logger.info(
                "Jackett autoconfig: discovered=%d
        configured_now=%d already=%d errors=%d",
                len(r.discovered_credentials),
                len(r.configured_now),
                len(r.already_present),
                len(r.errors),
            )
        except Exception:  # noqa: BLE001 — defense in depth
            logger.exception("Jackett autoconfig failed
        unexpectedly")
            app.state.jackett_autoconfig_last = None
    else:
        logger.info("Jackett autoconfig: skipped (no API key)")
        app.state.jackett_autoconfig_last = None
```

- [ ] **Step 2: Run unit tests to confirm no regression**

```
nice -n 19 ionice -c 3 -p 1 python3 -m pytest tests/unit/
        merge_service/test_jackett_autoconfig.py -v --import-
        mode=importlib
```

Expected: still 17 passed.

- [ ] **Step 3: Lint**

```
ruff check download-proxy/src/api/__init__.py
```

- [ ] **Step 4: Commit (local)**

```
git add download-proxy/src/api/__init__.py
git commit -m "feat(api): wire Jackett autoconfig into FastAPI
        lifespan"
```

Task 1.7: Add read-only endpoint `/api/v1/jackett/autoconfig/last`

Files: - Create: `download-proxy/src/api/jackett.py` - Modify: `download-proxy/src/api/__init__.py` (register router) - Modify: `tests/unit/merge_service/test_jackett_autoconfig.py` (add endpoint test)

☐ Step 1: Append failing test using FastAPI's TestClient

```
# Append to tests/unit/merge_service/test_jackett_autoconfig.py

def test_endpoint_returns_404_when_no_run_yet():
    from fastapi.testclient import TestClient

    from api import app

    # Force the state slot to absent.
    app.state.jackett_autoconfig_last = None
    with TestClient(app) as client:
        r = client.get("/api/v1/jackett/autoconfig/last")
    assert r.status_code == 404


def test_endpoint_returns_redacted_payload_when_run_present():
    from fastapi.testclient import TestClient
    from merge_service.jackett_autoconfig import AutoconfigResult

    from api import app

    app.state.jackett_autoconfig_last = AutoconfigResult(
        ran_at=datetime(2026, 4, 26, 14, 23, 11,
        tzinfo=timezone.utc),
        discovered_credentials=["RUTRACKER"],
        matched_indexers={"RUTRACKER": "rutracker"},
        configured_now=["rutracker"],
        already_present=[],
        skipped_no_match=[],
        skipped_ambiguous=[],
        errors=[],
    )
    with TestClient(app) as client:
        r = client.get("/api/v1/jackett/autoconfig/last")
    assert r.status_code == 200
    body = r.json()
    assert body["configured_now"] == ["rutracker"]
    text = r.text.lower()
    assert "password" not in text
    assert "cookie" not in text
```

☐ Step 2: Run — expect FAIL

```
nice -n 19 ionice -c 3 -p 1 python3 -m pytest tests/unit/
        merge_service/test_jackett_autoconfig.py -v --import-
        mode=importlib
```

Expected: 404 not raised correctly OR endpoint not found.

☐ Step 3: Create `api/jackett.py`

```
# download-proxy/src/api/jackett.py
"""Read-only endpoints exposing Jackett auto-configuration
        state."""

from fastapi import APIRouter, HTTPException, Request

router = APIRouter(tags=["jackett"])


@router.get("/jackett/autoconfig/last")
async def get_last_autoconfig(request: Request):
    last = getattr(request.app.state, "jackett_autoconfig_last",
        None)
    if last is None:
        raise HTTPException(status_code=404, detail="autoconfig
        has not run yet")
    return last.model_dump(mode="json", by_alias=True)
```

☐ Step 4: Register the router in `api/__init__.py`

Find the block at line 193-201 and add the new router. After `from .scheduler import router as scheduler_router  # noqa: E402,` add:

```
from .jackett import router as jackett_router  # noqa: E402
```

After `app.include_router(scheduler_router, prefix="/api/v1/schedules"),` add:

```
app.include_router(jackett_router, prefix="/api/v1")
```

☐ Step 5: Run — expect 19 passed

```
nice -n 19 ionice -c 3 -p 1 python3 -m pytest tests/unit/
        merge_service/test_jackett_autoconfig.py -v --import-
        mode=importlib
```

Expected: 19 passed.

☐ Step 6: Commit (local)

```
git add download-proxy/src/api/jackett.py download-proxy/src/api/
        __init__.py tests/unit/merge_service/
        test_jackett_autoconfig.py
git commit -m "feat(api): add /api/v1/jackett/autoconfig/last
        endpoint"
```

Task 1.8: Layer 1 gate — full unit suite green + push

- [ ] **Step 1: Run the full unit test directory to confirm no regression elsewhere**

```
nice -n 19 ionice -c 3 -p 1 python3 -m pytest tests/unit/ -v --
        import-mode=importlib --maxfail=1
```

Expected: all green. If anything else fails, halt and investigate (this is the “one fix attempt” rule).

- [ ] **Step 2: Push to all three remotes**

```
git push origin main && git push github main && git push upstream
        main
```

Expected: each push succeeds. If any rejects, halt — do not force.

Phase 2: Boot Real Stack (preparing for layers 2-7)

Task 2.1: Bring up clean stack

Files: none (operates on containers)

- [ ] **Step 1: Boot all services**

```
podman compose up -d
```

- [ ] **Step 2: Wait for healthchecks (timeout 3 min)**

```
for i in $(seq 1 36); do
  status=$(podman ps --format '{{.Names}} {{.Status}}' | grep -E
        'qbittorrent |jackett|qbittorrent-proxy ')
  if echo "$status" | grep -qi healthy && ! echo "$status" | grep
        -qi unhealthy; then
    echo "stack healthy after ${i}*5s"
    break
  fi
  sleep 5
done
podman ps --format '{{.Names}} {{.Status}}'
```

Expected: all 3 (or 4, including jackett) services show `(healthy)`. If any show `(unhealthy)` after 3 min, halt and dump logs:

```
podman logs qbittorrent-proxy --tail 200
podman logs jackett --tail 100
```

☐ **Step 3: Smoke-check the new endpoint**

```
curl -sf http://localhost:7187/api/v1/jackett/autoconfig/last |
        head -c 500 && echo
```

Expected: a JSON body (200) or `404` (autoconfig found no credentials, which is also valid). 5xx is a halt.

Phase 3: Layer 2 — Integration Tests (real Jackett)

Task 3.1: Write integration test against running Jackett

Files: - Create: `tests/integration/test_jackett_autoconfig_real.py`

☐ **Step 1: Write the test**

```
# tests/integration/test_jackett_autoconfig_real.py
"""Integration tests for jackett_autoconfig against a running
        Jackett.

Requires the full compose stack to be up. Skips if Jackett is
        unreachable.
"""
import os
import sys

sys.path.insert(
    0,
    os.path.join(os.path.dirname(__file__), "..", "..", "download-
        proxy", "src"),
)

import asyncio

import aiohttp
import pytest

JACKETT_URL = os.getenv("JACKETT_URL", "http://localhost:9117")
```

```
def _read_jackett_api_key() -> str:
    """Read the API key Jackett wrote to its config file."""
    path = os.path.join(
        os.getcwd(), "config", "jackett", "Jackett",
        "ServerConfig.json"
    )
    if not os.path.isfile(path):
        return ""
    import json

    with open(path) as f:
        data = json.load(f)
    return data.get("APIKey", "")


async def _jackett_alive(url: str, timeout: float = 2.0) -> bool:
    try:
        async with aiohttp.ClientSession() as s:
            async with s.get(f"{url}/UI/Login",
        timeout=aiohttp.ClientTimeout(total=timeout)) as r:
                return r.status < 500
    except Exception:
        return False


pytestmark = pytest.mark.asyncio


@pytest.fixture(scope="module")
def jackett_ready():
    if not asyncio.run(_jackett_alive(JACKETT_URL)):
        pytest.skip("Jackett unreachable — start the stack first")
    key = _read_jackett_api_key()
    if not key:
        pytest.skip("Jackett API key not yet generated")
    return key


async def
        test_autoconfig_runs_against_real_jackett(jackett_ready):
    from merge_service.jackett_autoconfig import
        autoconfigure_jackett

    # Inject a known credential triple in a synthetic env. Use a
        name
    # likely to fuzzy-match an indexer Jackett ships with —
        "1337x" is
    # a public indexer Jackett includes by default.
    env = {
        "1337X_USERNAME": "throwaway",
        "1337X_PASSWORD": "throwaway",
```

```
        "JACKETT_AUTOCONFIG_EXCLUDE":
            "QBITTORRENT,JACKETT,WEBUI,PROXY,MERGE,BRIDGE",
    }
    result = await autoconfigure_jackett(
        jackett_url=JACKETT_URL,
        api_key=jackett_ready,
        env=env,
        timeout=10.0,
    )
    # We can't guarantee 1337x is in this Jackett build, but the
        call
    # MUST succeed structurally — no errors that indicate a code
        bug.
    bug_indicating = [
        e for e in result.errors
        if "unexpected:" in e or "catalog_parse_failed" in e
    ]
    assert bug_indicating == [], result.errors


async def test_autoconfig_idempotent_on_second_run(jackett_ready):
    from merge_service.jackett_autoconfig import
        autoconfigure_jackett

    env = {
        "1337X_USERNAME": "throwaway",
        "1337X_PASSWORD": "throwaway",
    }
    r1 = await autoconfigure_jackett(
        jackett_url=JACKETT_URL, api_key=jackett_ready, env=env,
        timeout=10.0
    )
    r2 = await autoconfigure_jackett(
        jackett_url=JACKETT_URL, api_key=jackett_ready, env=env,
        timeout=10.0
    )
    # Anything configured by r1 should appear in r2's
        already_present.
    if r1.configured_now:
        for indexer_id in r1.configured_now:
            assert indexer_id in r2.already_present or indexer_id
        in r2.configured_now
```

☐

Step 2: Run

```
nice -n 19 ionice -c 3 -p 1 python3 -m pytest tests/integration/
        test_jackett_autoconfig_real.py -v --import-mode=importlib
```

Expected: 2 passed (or skipped if Jackett unreachable). If any fail with bug-indicating errors, fix and re-run once. If still failing, halt.

☐

Step 3: Commit

```
git add tests/integration/test_jackett_autoconfig_real.py
git commit -m
        "test(jackett_autoconfig): integration tests against real
        Jackett"
```

Task 3.2: Layer 2 gate — push

☐

Step 1: Push

```
git push origin main && git push github main && git push upstream
        main
```

Phase 4: Layer 3 — E2E Test (full stack from zero)

Task 4.1: Write E2E test

Files: - Create: `tests/e2e/test_jackett_autoconfig_e2e.py`

☐

Step 1: Write the test

```
# tests/e2e/test_jackett_autoconfig_e2e.py
"""E2E: clean-slate boot → autoconfig runs → search returns
        Jackett results."""
import os
import shutil
import subprocess
import time

import pytest
import requests

PROJECT_ROOT =
        os.path.abspath(os.path.join(os.path.dirname(__file__),
        "..", ".."))
JACKETT_CONFIG = os.path.join(PROJECT_ROOT, "config", "jackett")
MERGE_BASE = os.getenv("MERGE_SERVICE_URL", "http://
        localhost:7187")


def _have_compose() -> bool:
    return shutil.which("podman") is not None or
        shutil.which("docker") is not None
```

```
def _runtime() -> str:
    return "podman" if shutil.which("podman") else "docker"


@pytest.fixture(scope="module")
def clean_stack():
    if not _have_compose():
        pytest.skip("podman/docker not available")
    rt = _runtime()
    subprocess.run([rt, "compose", "down", "--remove-orphans"],
        cwd=PROJECT_ROOT, check=False)
    if os.path.isdir(JACKETT_CONFIG):
        shutil.rmtree(JACKETT_CONFIG)
    subprocess.run([rt, "compose", "up", "-d"], cwd=PROJECT_ROOT,
        check=True)
    # Wait for merge service.
    deadline = time.monotonic() + 180
    while time.monotonic() < deadline:
        try:
            r = requests.get(f"{MERGE_BASE}/health", timeout=2)
            if r.ok:
                break
        except requests.RequestException:
            pass
        time.sleep(5)
    else:

        pytest.fail("merge service did not become healthy within 3
        min")
    yield
    # Leave stack up for subsequent layers.


def test_autoconfig_endpoint_responds_after_clean_boot(clean_stack):
    deadline = time.monotonic() + 60
    while time.monotonic() < deadline:
        r = requests.get(f"{MERGE_BASE}/api/v1/jackett/autoconfig/
        last", timeout=5)
        if r.status_code in (200, 404):
            break
        time.sleep(2)
    else:
        pytest.fail("autoconfig endpoint never settled")

    if r.status_code == 200:
        body = r.json()
        assert "ran_at" in body
        assert "configured_now" in body


def test_search_through_merge_service_works(clean_stack):
    """Search should at minimum not 5xx; Jackett-sourced results
        may or
```

```
    may not appear depending on which indexers were configured."""
    r = requests.post(
        f"{MERGE_BASE}/api/v1/search",
        json={"query": "ubuntu", "category": "all"},
        timeout=10,
    )
    assert r.status_code < 500, r.text
```

☐ **Step 2: Run**

```
nice -n 19 ionice -c 3 -p 1 python3 -m pytest tests/e2e/
        test_jackett_autoconfig_e2e.py -v --import-mode=importlib
```

Expected: 2 passed. Hard stop on any 5xx or hang past 3 min.

☐ **Step 3: Commit + push (Layer 3 gate)**

```
git add tests/e2e/test_jackett_autoconfig_e2e.py
git commit -m "test(jackett_autoconfig): E2E clean-slate boot
        test"
git push origin main && git push github main && git push upstream
        main
```

Phase 5: Layer 4 — Security/Penetration Tests

Task 5.1: Credential-leak tests + bandit

Files: - Create: `tests/security/test_jackett_autoconfig_secrets.py`

☐ **Step 1: Write the test**

```
# tests/security/test_jackett_autoconfig_secrets.py
"""Verify Jackett autoconfig never leaks credentials anywhere."""
import io
import logging
import os
import subprocess
import sys

import pytest
import requests

sys.path.insert(
    0,
```

```
    os.path.join(os.path.dirname(__file__), "..", "..", "download-
        proxy", "src"),
)

PROJECT_ROOT =
        os.path.abspath(os.path.join(os.path.dirname(__file__),
        "..", ".."))
MERGE_BASE = os.getenv("MERGE_SERVICE_URL", "http://
        localhost:7187")
SENTINEL_PASSWORD = "p@ssw0rd_DO_NOT_LEAK_4f8a"
SENTINEL_COOKIE = "leak_canary_8b2c=evidence"


def test_endpoint_response_contains_no_sentinel_credentials():
    r = requests.get(f"{MERGE_BASE}/api/v1/jackett/autoconfig/
        last", timeout=5)
    if r.status_code == 404:
        pytest.skip("autoconfig has not run")
    body = r.text
    assert SENTINEL_PASSWORD not in body
    assert SENTINEL_COOKIE not in body


@pytest.mark.asyncio
async def
        test_traceback_from_forced_failure_excludes_credentials():
    from merge_service.jackett_autoconfig import
        autoconfigure_jackett

    log_stream = io.StringIO()
    handler = logging.StreamHandler(log_stream)
    handler.setLevel(logging.DEBUG)
    root = logging.getLogger()
    root.addHandler(handler)
    try:
        result = await autoconfigure_jackett(
            jackett_url="http://127.0.0.1:1",  # connection
        refused
            api_key=SENTINEL_PASSWORD,
        # use sentinel so we'd see it if leaked
            env={
                "TESTRACKER_USERNAME": "u",
                "TESTRACKER_PASSWORD": SENTINEL_PASSWORD,
                "TESTRACKER_COOKIES": SENTINEL_COOKIE,
            },
            timeout=1.0,
        )
    finally:
        root.removeHandler(handler)
    captured_logs = log_stream.getvalue()
    assert SENTINEL_PASSWORD not in captured_logs, "password
        leaked into logs"
    assert SENTINEL_COOKIE not in captured_logs, "cookie leaked
        into logs"
```

```
    assert SENTINEL_PASSWORD not in repr(result)
    assert SENTINEL_COOKIE not in repr(result)


def test_jackett_log_file_does_not_contain_sentinels():
    """If sentinel creds were posted to Jackett, they must not
        surface in its log."""
    log_path = os.path.join(PROJECT_ROOT, "config", "jackett",
        "Jackett", "log.txt")
    if not os.path.isfile(log_path):
        pytest.skip("Jackett log not present")
    with open(log_path, encoding="utf-8", errors="ignore") as f:
        content = f.read()
    assert SENTINEL_PASSWORD not in content
    assert SENTINEL_COOKIE not in content


def test_bandit_scan_module_clean():
    """Run bandit against the autoconfig module — zero high-
        severity findings."""
    target = os.path.join(
        PROJECT_ROOT, "download-proxy", "src", "merge_service",
        "jackett_autoconfig.py"
    )
    result = subprocess.run(
        ["bandit", "-q", "-f", "json", "-l", target],
        capture_output=True,
        text=True,
        check=False,
    )
    import json

    data = json.loads(result.stdout) if result.stdout else
        {"results": []}
    high = [r for r in data.get("results", []) if
        r.get("issue_severity") == "HIGH"]
    assert high == [], f"bandit HIGH findings: {high}"
```

☐ **Step 2: Run**

```
nice -n 19 ionice -c 3 -p 1 python3 -m pytest tests/security/
        test_jackett_autoconfig_secrets.py -v --import-
        mode=importlib
```

Expected: 4 passed (some may skip if stack not up).

☐ **Step 3: Commit + push (Layer 4 gate)**

```
git add tests/security/test_jackett_autoconfig_secrets.py
git commit -m "test(jackett_autoconfig): security tests + bandit
    scan"
git push origin main && git push github main && git push upstream
    main
```

Phase 6: Layer 5 — Benchmark

Task 6.1: Benchmark + baseline

Files: - Create: `tests/benchmark/test_jackett_autoconfig_perf.py` - Create: `tests/benchmark/baselines/jackett_autoconfig.json`

☐

Step 1: Write the benchmark test

```
# tests/benchmark/test_jackett_autoconfig_perf.py
"""Performance baselines for jackett_autoconfig."""
import os
import sys

sys.path.insert(
    0,
    os.path.join(os.path.dirname(__file__), "..", "..", "download-
        proxy", "src"),
)

import pytest


def test_env_scan_throughput(benchmark):
    from merge_service.jackett_autoconfig import
        _scan_env_credentials

    big_env = {f"VAR_{i}": str(i) for i in range(1000)}
    big_env.update({
        "RUTRACKER_USERNAME": "u",
        "RUTRACKER_PASSWORD": "p",
        "KINOZAL_USERNAME": "u",
        "KINOZAL_PASSWORD": "p",
    })

    result = benchmark(lambda: _scan_env_credentials(big_env,
        exclude=set()))
    assert "RUTRACKER" in result
    assert benchmark.stats["mean"] < 0.05  # <50ms hard floor


def test_fuzzy_match_throughput(benchmark):
    from merge_service.jackett_autoconfig import _match_indexers
```

```
    catalog = [{"id": f"indexer{i}", "name": f"Indexer{i}"} for i
        in range(50)]
    catalog.append({"id": "rutracker", "name": "RuTracker"})
    bundles = {f"TRACKER{i}": {"username": "u", "password": "p"}
        for i in range(10)}
    bundles["RUTRACKER"] = {"username": "u", "password": "p"}

    result = benchmark(lambda: _match_indexers(bundles, catalog,
        override={}))
    assert benchmark.stats["mean"] < 0.20  # <200ms hard floor


def test_full_autoconfigure_with_unreachable_jackett(benchmark):
    """Benchmark the failure-fast path. Sync test wraps async via
        asyncio.run()."""
    import asyncio

    from merge_service.jackett_autoconfig import
        autoconfigure_jackett

    async def run():
        return await autoconfigure_jackett(
            jackett_url="http://127.0.0.1:1",
            api_key="fake",
            env={"RUTRACKER_USERNAME": "u", "RUTRACKER_PASSWORD":
        "p"},
            timeout=1.0,
        )

    def runner():
        return asyncio.run(run())

    runner()  # warmup
    result = benchmark(runner)
    assert result.errors  # we expect an error (unreachable)
    assert benchmark.stats["mean"] < 2.0  # cap is timeout +
        overhead
```

- [ ] **Step 2: Run, capturing JSON output**

```
mkdir -p tests/benchmark/baselines
nice -n 19 ionice -c 3 -p 1 python3 -m pytest tests/benchmark/
        test_jackett_autoconfig_perf.py \
  -v --import-mode=importlib \
  --benchmark-json=tests/benchmark/baselines/
        jackett_autoconfig.json
```

Expected: 3 passed.

- [ ] **Step 3: Sanity-check the baseline file is non-empty**

```
test -s tests/benchmark/baselines/jackett_autoconfig.json && echo
    "baseline OK"
```

▣

Step 4: Commit + push (Layer 5 gate)

```
git add tests/benchmark/test_jackett_autoconfig_perf.py tests/
    benchmark/baselines/jackett_autoconfig.json
git commit -m "test(jackett_autoconfig): benchmark + baseline"
git push origin main && git push github main && git push upstream
    main
```

Phase 7: Layer 6 — Contract / Automation

Task 7.1: Schemathesis contract test

Files: - Create: `tests/contract/test_jackett_autoconfig_contract.py`

▣

Step 1: Write the test

```
# tests/contract/test_jackett_autoconfig_contract.py
"""Schemathesis-driven contract tests for /api/v1/jackett/
    autoconfig/last."""
import os

import pytest

MERGE_BASE = os.getenv("MERGE_SERVICE_URL", "http://
    localhost:7187")


def test_autoconfig_endpoint_contract():
    """Verify the endpoint adheres to its OpenAPI schema."""
    schemathesis = pytest.importorskip("schemathesis")

    schema = schemathesis.openapi.from_url(f"{MERGE_BASE}/
        openapi.json")

    # Filter schemathesis to just our endpoint to keep runtime
        bounded.
    @schema.parametrize(endpoint="/api/v1/jackett/autoconfig/
        last")
    @schemathesis.checks([schemathesis.checks.not_a_server_error,
        schemathesis.checks.response_schema_conformance])
    def _check(case):
        case.call_and_validate()
```

```
    _check()
```

Note: Schemathesis APIs vary across versions. If `from_url` / `parametrize` are wrong for the installed version, the test should be adjusted to match. Run once and fix on first failure.

▣ **Step 2: Run**

```
nice -n 19 ionice -c 3 -p 1 python3 -m pytest tests/contract/
        test_jackett_autoconfig_contract.py -v --import-
        mode=importlib
```

Expected: passed (or skipped if schemathesis missing).

▣ **Step 3: Commit + push (Layer 6 gate)**

```
git add tests/contract/test_jackett_autoconfig_contract.py
git commit -m "test(jackett_autoconfig): contract test via
        schemathesis"
git push origin main && git push github main && git push upstream
        main
```

Phase 8: Layer 7 — Challenge Script

Task 8.1: Write the clean-slate Challenge

Files: - Create: `challenges/scripts/jackett_autoconfig_clean_slate.sh` (executable) - Create: `challenges/scripts/run_all_challenges.sh` (executable)

▣ **Step 1: Create the challenges directory and primary script**

```
mkdir -p challenges/scripts

# challenges/scripts/jackett_autoconfig_clean_slate.sh
#!/usr/bin/env bash
# CONST-032 regression guard: clean-slate Jackett autoconfig flow.
#
# 1. Tear down stack
# 2. Wipe ./config/jackett
# 3. Boot stack
# 4. Wait for healthchecks (3 min ceiling)
# 5. Poll /api/v1/jackett/autoconfig/last until populated
# 6. Assert configured_now >= 1 OR (creds in env exist AND
        skipped_no_match accounts for them)
```

```
# 7. Run a search and assert no 5xx
set -euo pipefail

PROJECT_ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/../.." && pwd)"
cd "$PROJECT_ROOT"

RUNTIME="${CONTAINER_RUNTIME:-podman}"
if ! command -v "$RUNTIME" >/dev/null 2>&1; then
  RUNTIME="docker"
fi
MERGE="${MERGE_SERVICE_URL:-http://localhost:7187}"

step() { echo ">>> $*"; }

step "1. Tear down"
"$RUNTIME" compose down --remove-orphans || true

step "2. Wipe ./config/jackett"
rm -rf ./config/jackett

step "3. Boot stack"
"$RUNTIME" compose up -d

step "4. Wait for /health (3 min)"
for i in $(seq 1 36); do
  if curl -sf "$MERGE/health" >/dev/null 2>&1; then
    echo "   merge service healthy after $((i*5))s"
    break
  fi
  sleep 5
done
if ! curl -sf "$MERGE/health" >/dev/null 2>&1; then
  echo "FAIL: merge service unhealthy after 3 min"
  "$RUNTIME" logs qbittorrent-proxy --tail 200 || true
  exit 1
fi

step "5. Poll autoconfig endpoint (60s)"
deadline=$(( $(date +%s) + 60 ))
status=0
body=""
while [ "$(date +%s)" -lt "$deadline" ]; do
  http_code=$(curl -s -o /tmp/autoconfig.json -w '%{http_code}'
        "$MERGE/api/v1/jackett/autoconfig/last" || true)
  if [ "$http_code" = "200" ]; then
    body=$(cat /tmp/autoconfig.json)
    status=200
    break
  elif [ "$http_code" = "404" ]; then
    status=404
    break
  fi
```

```
  sleep 2
done

step "6. Assert response shape"
if [ "$status" = "404" ]; then
  echo
        "   autoconfig has no recorded run — acceptable if no
        creds in env"
else
  echo "$body" | python3 -m json.tool >/dev/null || {
    echo "FAIL: autoconfig body is not valid JSON"
    exit 1
  }
  configured_now=$(echo "$body" | python3 -c "import json,sys;
        print(len(json.load(sys.stdin).get('configured_now',
        [])))")
  echo "   configured_now count: $configured_now"
fi

step "7. Run a search; assert no 5xx"
search_code=$(curl -s -o /tmp/search.json -w '%{http_code}' \
  -X POST "$MERGE/api/v1/search" \
  -H 'Content-Type: application/json' \
  -d '{"query":"ubuntu","category":"all"}' || true)
if [ "$search_code" -ge 500 ]; then
  echo "FAIL: search returned $search_code"
  cat /tmp/search.json
  exit 1
fi
echo "   search returned $search_code — OK"

echo "PASS: jackett_autoconfig_clean_slate"

# challenges/scripts/run_all_challenges.sh
#!/usr/bin/env bash
# Runs every challenge script in challenges/scripts/, fails fast.
set -euo pipefail
HERE="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
fails=0
for script in "$HERE"/*.sh; do
  name="$(basename "$script")"
  [ "$name" = "run_all_challenges.sh" ] && continue
  echo "=== $name ==="
  if ! "$script"; then
    echo "FAIL: $name"
    fails=$((fails+1))
  fi
done
echo "challenges total fails: $fails"
exit $fails
```

```
chmod +x challenges/scripts/jackett_autoconfig_clean_slate.sh
        challenges/scripts/run_all_challenges.sh
```

- [ ] **Step 2: Run the challenge**

```
./challenges/scripts/jackett_autoconfig_clean_slate.sh
```

Expected: prints `PASS: jackett_autoconfig_clean_slate` and exits 0.

- [ ] **Step 3: Commit + push (Layer 7 gate)**

```
git add challenges/scripts/jackett_autoconfig_clean_slate.sh
        challenges/scripts/run_all_challenges.sh
git commit -m "test(jackett_autoconfig): clean-slate challenge
        regression guard"
git push origin main && git push github main && git push upstream
        main
```

Phase 9: Parity Audit Deliverable

Task 9.1: Author `docs/migration/PARITY_GAPS.md`

Files: - Create: `docs/migration/PARITY_GAPS.md`

- [ ] **Step 1: Read each Python module and find its Go counterpart**

For each module, run side-by-side:

```
# Per Python module of interest:
ls download-proxy/src/merge_service/  # then Read each
ls qBitTorrent-go/internal/service/    # then Read each
ls download-proxy/src/api/             # then Read each
ls qBitTorrent-go/internal/api/        # then Read each
```

Build a table by direct read. **No code changes** during this phase.

- [ ] **Step 2: Write the audit document**

```
# Python → Go Parity Gaps

Last audited: 2026-04-26 (commit `<sha from /tmp/jackett-
        autoconfig-baseline-sha.txt>`)
Audit method: side-by-side read of public API surface; no behavior
        testing performed.
```

```
## Summary

- Total Python features audited: <N>
- Fully ported: <X>
- Partial: <Y>
- Missing: <Z>

## Status definitions

- **Ported** — feature exists in Go with equivalent behavior.
- **Partial** — feature exists in Go but lacks a sub-behavior.
      Note the gap inline.
- **Missing** — feature has no Go counterpart.

## Matrix

| Python module | Feature | Go location | Status
      | Risk if Go-only today |
|---|---|---|---|---|
| `merge_service/search.py` | `start_search()` orchestrator |
      `internal/service/merge_search.go` | <fill> | <fill> |
| `merge_service/search.py` | `_classify_plugin_stderr()` error
      categorization | <fill> | <fill> | <fill> |
| `merge_service/deduplicator.py` | tiered dedup (infohash →
      name+size → fuzzy) | <fill> | <fill> | <fill> |
| `merge_service/enricher.py` | TMDB title resolution | <fill> |
      <fill> | <fill> |
| `merge_service/enricher.py` | OMDb fallback | <fill> | <fill> |
      <fill> |
| `merge_service/enricher.py` | TVMaze / AniList / MusicBrainz /
      OpenLibrary | <fill> | <fill> | <fill> |
| `merge_service/validator.py` | BEP 48 HTTP scrape | <fill> |
      <fill> | <fill> |
| `merge_service/validator.py` | BEP 15 UDP scrape | <fill> |
      <fill> | <fill> |
| `merge_service/scheduler.py` | recurring search scheduling |
      <fill> | <fill> | <fill> |
| `merge_service/hooks.py` | hook callbacks | <fill> | <fill> |
      <fill> |
| `merge_service/retry.py` | shared retry policy | <fill> |
      <fill> | <fill> |
| `merge_service/jackett_autoconfig.py` | env-discovery + indexer
      config | (none) | Missing | Go boot will not auto-
      configure Jackett — manual UI required |
| `api/routes.py` | merge/search REST | `internal/api/` | <fill>
      | <fill> |
| `api/auth.py` | private-tracker auth flows | <fill> | <fill> |
      <fill> |
| `api/hooks.py` | hooks endpoints | <fill> | <fill> | <fill> |
| `api/scheduler.py` | scheduler endpoints | <fill> | <fill> |
      <fill> |
| `api/jackett.py` | autoconfig read endpoint | (none) | Missing
      | Same as above |
| `ui/...` | Jinja2 dashboard | <fill> | <fill> | <fill> |
```

```
## Per-gap follow-up specs (proposed, in priority order)

1. <fill> — biggest user-visible gap
2. <fill>
3. <fill>
```

Replace every `<fill>` and `<sha>` token by reading the actual Go modules.

☐ **Step 3: Sanity-check no `<fill>` tokens remain**

```
grep -n '<fill>' docs/migration/PARITY_GAPS.md && echo
        "INCOMPLETE" || echo "audit complete"
```

Expected: `audit complete`.

☐ **Step 4: Commit + push**

```
git add docs/migration/PARITY_GAPS.md
git commit -m "docs(migration): parity audit Python→Go
        (PARITY_GAPS.md)"
git push origin main && git push github main && git push upstream
        main
```

Phase 10: Documentation Updates

Task 10.1: Update CLAUDE.md, AGENTS.md, JACKETT_INTEGRATION.md, .env.example

Files: - Modify: `CLAUDE.md` - Modify: `AGENTS.md` - Modify: `docs/JACKETT_INTEGRATION.md` - Modify: `.env.example` (if exists)

☐ **Step 1: Update `CLAUDE.md` env vars line**

Find the existing `## Environment Variables` section (around line 173). Append `JACKETT_INDEXER_MAP`, `JACKETT_AUTOCONFIG_EXCLUDE` to the “Key” list. Append a one-line note about the new endpoint:

```
*Jackett auto-configuration:* at proxy startup, the merge service
        discovers
`<NAME>_USERNAME/_PASSWORD/_COOKIES` env triples and configures
        matching
Jackett indexers. Override fuzzy matches with
        `JACKETT_INDEXER_MAP=NAME:indexer_id,...`.
Exclude internal prefixes with
        `JACKETT_AUTOCONFIG_EXCLUDE=PREFIX,...` (defaults
```

```
to `QBITTORRENT,JACKETT,WEBUI,PROXY,MERGE,BRIDGE`). Last-run
        summary at
`GET /api/v1/jackett/autoconfig/last` (redacted).
```

☐ **Step 2: Update `AGENTS.md` with the same env additions**

Find the env-vars table or section in `AGENTS.md` and append the same two vars + endpoint note. Match `AGENTS.md`'s formatting.

☐ **Step 3: Append "Auto-Configuration" section to `docs/JACKETT_INTEGRATION.md`**

Append a section explaining: - Discovery algorithm (env scan → fuzzy match → idempotent POST) - Override and exclude env vars - Endpoint shape with example response - Failure modes (best-effort, never blocks boot)

☐ **Step 4: Update `.env.example` if present**

```
test -f .env.example && {
  grep -q '^JACKETT_INDEXER_MAP=' .env.example || echo
        'JACKETT_INDEXER_MAP=' >> .env.example
  grep -q '^JACKETT_AUTOCONFIG_EXCLUDE=' .env.example || echo
        'JACKETT_AUTOCONFIG_EXCLUDE=' >> .env.example
}
```

☐ **Step 5: Commit + push**

```
git add CLAUDE.md AGENTS.md docs/
        JACKETT_INTEGRATION.md .env.example 2>/dev/null
git commit -m "docs(jackett_autoconfig): document env vars +
        endpoint"
git push origin main && git push github main && git push upstream
        main
```

Phase 11: Final Clean-Slate Verification + Handoff

Task 11.1: Re-run the full clean-slate path one more time

Files: none

☐

Step 1: Tear down + wipe Jackett state

```
podman compose down --remove-orphans
rm -rf ./config/jackett
```

☐

Step 2: Run the challenge end-to-end

```
./challenges/scripts/jackett_autoconfig_clean_slate.sh 2>&1 |
        tee /tmp/final-clean-slate.log
```

Expected: ends with `PASS: jackett_autoconfig_clean_slate`. If not, halt — this is a final guard.

☐

Step 3: Confirm full unit + integration suites still green

```
nice -n 19 ionice -c 3 -p 1 python3 -m pytest tests/unit/ tests/
        integration/ -v --import-mode=importlib --maxfail=1
```

☐

Step 4: Capture final state into a verification commit

```
git commit --allow-empty -m "chore(jackett_autoconfig): final
        clean-slate verification

$(tail -40 /tmp/final-clean-slate.log)
"
git push origin main && git push github main && git push upstream
        main
```

Task 11.2: Handoff message

☐

Step 1: Confirm endpoints + state

```
echo "==== container status ===="
podman ps --format '{{.Names}} {{.Status}}'
echo
echo "==== /health ===="
curl -sf http://localhost:7187/health
echo
echo "==== /api/v1/jackett/autoconfig/last ===="
curl -s http://localhost:7187/api/v1/jackett/autoconfig/last |
        python3 -m json.tool
```

☐

Step 2: Print handoff summary to the operator

Surface to the operator: - Stack is up at 7185 (qBit), 7186 (proxy), 7187 (merge), 9117 (Jackett). - All 7 test layers + clean-slate challenge are green. - Parity audit at `docs/migration/PARITY_GAPS.md`. - Last commits pushed to `origin`, `github`, `upstream` (same URL). - Ready for manual testing.

Self-Review

This plan was checked against the spec on 2026-04-26:

- §1 Problem & Intent → Phases 0-11 cover all three intent threads (rebuild, auto-config, tests + parity).
- §2 Decisions Locked → all 7 Q&A choices reflected in concrete tasks (denylist in 1.2, override precedence in 1.3, idempotency in 1.4, 60s ceiling in 1.5, FastAPI lifespan call site in 1.6, redacted endpoint in 1.7, per-layer push gate after every phase).
- §3 Architecture → tasks 1.1-1.7 implement exactly the §3.1-3.5 surface; Section 4 data-flow honored.
- §4 Data flow → lifespan call (Task 1.6) + endpoint (Task 1.7) match diagram.
- §5 Test Matrix → one phase per layer (1, 3, 4, 5, 6, 7, 8). Resource caps (`nice -n 19 ionice -c 3 -p 1`) on every pytest invocation.
- §6 Error handling → all 11 failure rows covered by Task 1.5 orchestrator branches + tests in 1.4-1.5 + security tests in 5.1.
- §7 Parity audit → Phase 9.
- §8 Verification & Push → per-layer push gates after each phase + final empty commit in Task 11.1 captures the verification log.
- §9 File inventory → every file in §9.1/§9.2 has a corresponding task. (Plus `docs/issues/fixed/BUGFIXES.md` per §9.4 — only created if a bug surfaces during implementation.)
- §10 Out of scope → Go-side anything explicitly excluded.
- §11 Open questions → Branch name not specified (assume `main`). Bugfix log creation deferred to first bug.

Minor deviation logged in plan header: endpoint path `/api/v1/jackett/autoconfig/last` instead of spec's `/merge/jackett/autoconfig/last`, for router-prefix consistency with existing `auth.py`/`hooks.py`/`scheduler.py`.

Hard stops (recap): - Layer fail after one fix attempt → halt, ask operator. - `git push` rejected → halt, do not force. - Healthcheck unhealthy after 3 min → halt, dump logs. - Bench regression > 2× baseline → halt, ask.